

Forward List (Homework)

1. Métodos de la Clase List (Forward List)

1.1. front():

Devuelve el valor del primer nodo de la lista. Simplemente accede al miembro data del nodo head. Si la lista está vacía, lanza una excepción para evitar errores. Su costo de tiempo es constante, $O(1)$.

Código:

```
T front(){
    if (empty()) {
        throw out_of_range("La lista está vacía.");
    }
    return head->data;
}
```

Funcionamiento:

Primero, agregamos los nodos a la lista con push_front.

```
List l = List<int>();

cout << "Agregando elementos: 6, 2, 9, 0, 3 (usando push_front)" << endl;
l.push_front( data: 6);
l.push_front( data: 2);
l.push_front( data: 9);
l.push_front( data: 0);
l.push_front( data: 3);

cout << "Elemento al frente: " << l.front() << endl;
```

Resultado:

Elemento al frente: 3

El valor del primer elemento, 3, es retornado y mostrado en la consola.

1.2. back():

Se retorna el valor del último nodo de la lista. Para hacerlo, recorre la lista desde el inicio hasta encontrar el nodo cuyo puntero next es nullptr, lo que indica que es el último. Su costo de tiempo es lineal, $O(n)$, ya que necesita iterar por todos los elementos.

Código:

```

T back(){
    if (empty()) {
        throw out_of_range("La lista está vacía.");
    }
    Node *temp = head;
    while(temp->next != nullptr){
        temp = temp->next;
    }
    return temp->data;
}

```

Funcionamiento :

```
cout << "Elemento al final: " << l.back() << endl;
```

Resultado:

Elemento al final: 6

El valor del último elemento, 6, es retornado y mostrado.

1.3. push_front():

El método inserta un nuevo nodo al comienzo de la lista. Para hacerlo, se crea un nuevo nodo, se hace que su puntero next apunte al nodo que actualmente es la cabeza (head), y luego se actualiza el head de la lista para que sea el nuevo nodo. Esta operación es muy eficiente, con un costo de tiempo constante, $O(1)$.

Código:

```

void push_front(int data){
    Node *new_node = new Node(data);
    new_node->next = head;
    head = new_node;
    current_size++;
}

```

Funcionamiento:

```

cout << "Agregando elementos: 6, 2, 9, 0, 3 (usando push_front)" << endl;
l.push_front( data: 6);
l.push_front( data: 2);
l.push_front( data: 9);
l.push_front( data: 0);
l.push_front( data: 3);

cout << "Lista actualizada: ";
l.iterate();

```

Resultado:

Agregando elementos: 6, 2, 9, 0, 3 (usando push_front)
Lista actualizada: 3 0 9 2 6

Como se ve, cada nuevo elemento se inserta al inicio de la lista, invirtiendo el orden de inserción.

1.4. push_back():

Se añade un nuevo nodo al final de la lista. Se recorre la lista desde el head hasta encontrar el último nodo (aquel cuyo puntero next es nullptr). Una vez encontrado, se hace que su puntero next apunte al nuevo nodo. Esta operación requiere recorrer toda la lista, por lo que su costo de tiempo es lineal, $O(n)$, donde n es el número de elementos.

Código:

```
void push_back(int data){
    Node *new_node = new Node(data);
    if (empty()){
        head = new_node;
    } else {
        Node *temp = head;
        while(temp->next != nullptr){
            temp = temp->next;
        }
        temp->next = new_node;
    }
    current_size++;
}
```

Funcionamiento:

```
cout << "Agregando 10 al final (push_back)" << endl;
l.push_back( data: 10);
```

Resultado:

```
Agregando 10 al final (push_back)
Lista actualizada: 3 0 9 2 6 10
```

Se observa que el valor 10 se añade al final de la lista.

1.5. pop_front():

Remueve el primer nodo de la lista. Se guarda una referencia al nodo head, se actualiza el head para que sea el siguiente nodo en la lista, y finalmente se libera la memoria del nodo original que era el head. Su costo de tiempo es constante, $O(1)$.

Código:

```

void pop_front() {
    if (empty()) {
        throw out_of_range("La lista está vacía.");
    }
    Node *temp = head;
    head = head->next;
    delete temp;
    tamano--;
}

```

Funcionamiento:

```

cout << "Removiendo el primer elemento (pop_front)"<< endl;
l.pop_front();

```

Resultado:

Removiendo el primer elemento (pop_front)

Lista actualizada: 0 9 2 6 10

El valor 3, que era el primer elemento, es removido.

1.6. pop_back():

Elimina el último nodo de la lista. Requiere un recorrido para encontrar el penúltimo nodo, ya que su puntero next debe ser actualizado a nullptr. El último nodo es entonces eliminado. Similar a push_back, su costo de tiempo es lineal, $O(n)$.

Código:

```

void pop_back() {
    if (empty()) {
        throw out_of_range("La lista está vacía.");
    }
    if (head->next == nullptr) {
        delete head;
        head = nullptr;
        tamano--;
    }
    Node *temp = head;
    while (temp->next->next != nullptr) {
        temp = temp->next;
    }
    delete temp->next;
    temp->next = nullptr;
    tamano--;
}

```

Funcionamiento:

```
cout << "Removiendo el último elemento (pop_back): " << endl;
l.pop_back();
```

Resultado:

```
Removiendo el último elemento (pop_back):
Lista actualizada: 0 9 2 6
```

El valor 10, que era el último elemento, es removido.

1.7. **operator[]:**

accedemos al elemento en una posición específica de la lista enlazada, similar a un arreglo. Recorre la lista desde el head un número de veces igual al índice dado. Si el índice está fuera de los límites, se lanza una excepción. Su costo de tiempo es lineal, $O(n)$, porque en el peor caso (último elemento) debe recorrer toda la lista.

Código:

```
T operator[](int index){
    if (index < 0 || index >= current_size) {
        throw out_of_range("Índice fuera de rango.");
    }
    Node *temp = head;
    for(int i = 0; i < index; i++){
        temp = temp->next;
    }
    return temp->data;
}
```

Funcionamiento:

```
cout << "Elemento en la posición 2: " << l[2] << endl;
```

Resultado:

```
Elemento en la posición 2: 2
```

En la lista 0 9 2 6 10, el elemento en el índice 2 (tercer elemento, contando desde 0) es 9.

1.8. **empty():**

Se verifica si la lista está vacía. Simplemente comprueba si el puntero head es nullptr. Si es así, la lista no contiene nodos. Su costo de tiempo es constante, $O(1)$.

Código:

```
bool empty(){
    return head == nullptr;
}
```

Funcionamiento:

```
cout << "La lista está vacía? " << (l.empty() ? "Sí" : "No") << endl;
```

Resultado:

```
La lista está vacía? No
```

La lista 0 9 2 6 no está vacía, por lo que el resultado es No.

1.9. size():

Se devuelve el número total de elementos en la lista. Se mantiene una variable privada `current_size` que se incrementa en cada inserción (`push_front`, `push_back`) y se decrementa en cada eliminación (`pop_front`, `pop_back`). Esto evita tener que recorrer la lista para contar los nodos, lo que lo hace muy eficiente con un costo de tiempo constante, $O(1)$.

Código:

```
int size(){
    return current_size;
}
```

Funcionamiento:

```
cout << "Tamaño de la lista: " << l.size() << endl;
```

Resultado:

```
Tamaño de la lista: 5
```

Después de varias inserciones y eliminaciones, el tamaño de la lista se actualiza y se muestra como 5.

1.10. clear():

elimina todos los nodos de la lista. Utiliza un bucle que llama repetidamente a `pop_front()` hasta que la lista esté vacía. Esta es una forma segura de liberar la memoria de cada nodo. Su costo de tiempo es lineal, $O(n)$, ya que procesa cada elemento una vez.

Código:

```
void clear(){
    while(!empty()){
        pop_front();
    }
}
```

Funcionamiento:

```
cout << "Vaciando la lista (clear)..." << endl;
l.clear();
cout << "La lista está vacía? " << (l.empty() ? "Sí" : "No") << endl;
```

Resultado:

```
Vaciando la lista (clear)...
La lista está vacía? Sí
```

Después de la llamada a clear(), la lista queda vacía y empty() retorna true.

1.11. sort() (Insertion Sort):

Se implementa el algoritmo de **ordenamiento por inserción** (Insertion Sort) para listas enlazadas. Se construye una nueva lista ordenada a partir de los nodos de la original. Cada nodo de la lista original se toma y se inserta en su posición correcta en la nueva lista, manteniendo el orden. Este proceso es in-place (no se crean nuevos nodos). Su costo de tiempo en el peor caso es cuadrático, $O(n^2)$.

Código:

```
void sort(){
    if (empty() || head->next == nullptr) {
        return;
    }

    Node *sorted_head = nullptr;
    Node *current = head;

    while (current != nullptr) {
        Node *next_node = current->next;
        if (sorted_head == nullptr || current->data < sorted_head->data) {
            current->next = sorted_head;
            sorted_head = current;
        } else {
            Node *temp = sorted_head;
            while (temp->next != nullptr && temp->next->data < current->data) {
                temp = temp->next;
            }
            current->next = temp->next;
            temp->next = current;
        }
        current = next_node;
    }
    head = sorted_head;
}
```

Funcionamiento:

Removiendo el último elemento (pop_back):

Lista actualizada: 0 9 2 6

```
cout << "Ordenando la lista..." << endl;
l.sort();
```

Resultado:

Ordenando la lista...

Lista ordenada: 0 2 6 9

La lista desordenada se transforma en una lista con los elementos ordenados de forma ascendente.

1.12. reverse():

invierte el orden de los nodos en la lista. Lo hace reasignando los punteros next de cada nodo para que apunten al nodo anterior en lugar del siguiente. Utiliza tres punteros temporales (prev, current, y next_node) para lograrlo. Este proceso se realiza en una sola pasada, por lo que su costo de tiempo es lineal, $O(n)$.

Código:

```
void reverse(){
    Node *prev = nullptr;
    Node *current = head;
    Node *next_node = nullptr;

    while (current != nullptr){
        next_node = current->next;
        current->next = prev;
        prev = current;
        current = next_node;
    }
    head = prev;
}
```

Funcionamiento:

```
cout << "Revertiendo la lista..." << endl;
l.reverse();
cout << "Lista revertida: ";
l.iterate();
```


Resultado:

Revirtiendo la lista...

Lista revertida: 9 6 2 0

La lista ordenada se invierte para que el último elemento sea el primero y viceversa.

2. Ejercicios Adicionales

2.1. Invertir una Lista Enlazada (reverse()):

La función reverse() invierte la dirección de los punteros next de los nodos. Para ello, se utilizan tres punteros auxiliares: prev (el nodo anterior), current (el nodo actual) y next_node (el nodo siguiente, para no perder la referencia). En cada iteración, el puntero del nodo actual se reasigna para que apunte a prev, y los tres punteros se mueven un paso adelante.

Código:

```
void reverse(){
    Node *prev = nullptr;
    Node *current = head;
    Node *next_node = nullptr;

    while (current != nullptr){
        next_node = current->next;
        current->next = prev;
        prev = current;
        current = next_node;
    }
    head = prev;
}
```

2.2. Mezclar dos Listas Ordenadas (merge_sorted_lists()):

Para mezclar dos listas ordenadas en $O(n)$, se usa un enfoque de dos punteros. Se recorren ambas listas simultáneamente. Se compara el valor de los nodos actuales de cada lista y se añade el menor de los dos a una nueva lista de resultado. El puntero de la lista que aportó el nodo se avanza. Al final, se añaden los elementos restantes de la lista que no ha terminado.

Código:

```
List merge_sorted_lists(List &l1, List &l2){
    List merged_list;
    Node *p1 = l1.get_head();
    Node *p2 = l2.get_head();

    while(p1 != nullptr && p2 != nullptr){
```

```

        if (p1->data <= p2->data){
            merged_list.push_back(p1->data);
            p1 = p1->next;
        } else {
            merged_list.push_back(p2->data);
            p2 = p2->next;
        }
    }

    while(p1 != nullptr){
        merged_list.push_back(p1->data);
        p1 = p1->next;
    }

    while(p2 != nullptr){
        merged_list.push_back(p2->data);
        p2 = p2->next;
    }

    return merged_list;
}

```

2.3. Hallar la Intersección de dos Listas Ordenadas (intersection_of_sorted_lists()):

Similar a la fusión, este método utiliza dos punteros para encontrar elementos comunes en dos listas ordenadas en $O(n)$. Se comparan los elementos de los punteros.

- Si son iguales, el elemento es parte de la intersección. Se añade a una nueva lista de resultado y se avanzan ambos punteros.
- Si son diferentes, se avanza solo el puntero del nodo con el valor más pequeño, ya que no puede ser parte de la intersección con el nodo actual de la otra lista.

Código:

```

List intersection_of_sorted_lists(List &l1, List &l2){
    List intersection_list;
    Node *p1 = l1.get_head();
    Node *p2 = l2.get_head();

    while(p1 != nullptr && p2 != nullptr){
        if (p1->data == p2->data){
            intersection_list.push_back(p1->data);
            p1 = p1->next;
            p2 = p2->next;
        } else if (p1->data < p2->data){
            p1 = p1->next;
        } else {

```

```

        p2 = p2->next;
    }
}
return intersection_list;
}
}

```

2.4. Insertion Sort en Listas Enlazadas (sort()):

El algoritmo de Insertion Sort para listas enlazadas construye una lista ordenada (sorted_head) a partir de la lista original. Se recorre la lista original (current) nodo por nodo. Cada nodo se "desvincula" de la lista original y se inserta en la posición correcta de la lista ordenada. Esto implica buscar la posición de inserción en la lista ordenada (sorted_head) y luego reajustar los punteros. Su costo de tiempo es $O(n^2)$ en el peor caso.